MindAnchor

Codebase Explained

Business & Functional explanation of every file

Generated: 2026-06-12

Layer	Technology
Backend API	Python · FastAPI · SQLAlchemy · Alembic
Database	PostgreSQL (Render managed)
AI	Anthropic Claude (Opus for planning, Sonnet for speed)
Frontend	React · TypeScript · Vite · Tailwind CSS
Auth	bcrypt password · JWT tokens
Hosting	Render (backend) · Vercel (frontend)
Mobile	PWA (installable) · Expo scaffold (future)

Table of Contents

SECTION 1 — BACKEND: CORE FILES

■ core/config.py

● Business Layer

The single place where all environment-specific settings live. Instead of hardcoding URLs or secrets anywhere in the codebase, every value that can differ between your laptop and the production server is declared here and read from environment variables. Think of it as the app's control panel.

● Functional Layer

Uses pydantic-settings BaseSettings which automatically reads values from .env file (local dev) or real environment variables (Render production). Key settings: database_url (PostgreSQL connection string), jwt_secret (signs/verifies login tokens), jwt_expire_minutes (7 days default), password_hash (bcrypt hash of owner password), anthropic_api_key (empty = offline stub mode), cors_origins (list of allowed frontend URLs). get_settings() is decorated with @lru_cache so the file is only parsed once per process, not on every request.

■ auth.py

● Business Layer

The lock on the front door. Without a valid login, no data can be read or written. Implements the standard 'password → token' pattern: you prove who you are once with a password, and then carry a short-lived digital pass (JWT) for all subsequent requests.

● Functional Layer

verify_password(plain, hashed) uses passlib/bcrypt constant-time comparison so timing attacks can't guess the password. create_access_token() builds a JWT payload {sub: 'owner', exp: 7 days} and signs it with HS256 using JWT_SECRET. get_current_user is a FastAPI dependency that extracts the Authorization: Bearer token header, decodes and validates the JWT, raises HTTP 401 on any failure. Applied at the router level in main.py so no route can accidentally be left unprotected. Uses python-jose for JWT operations.

■ db.py

● Business Layer

Manages the connection to the PostgreSQL database. Deliberately 'lazy' — it never tries to connect when the Python module is imported (which would crash tests or local runs without a database). The connection only happens when the first real request arrives.

● Functional Layer

get_engine() decorated with @lru_cache creates the SQLAlchemy engine from settings.database_url only once. SessionLocal is the SQLAlchemy session factory. get_db() is a FastAPI dependency that opens a session, yields it to the route handler, then closes it via finally — ensuring every request gets a clean session and no connections are leaked. init_db() creates all tables from Base.metadata (used by dev script; Alembic used in production).

■ models.py

● Business Layer

Defines the shape of all data stored in the database. Think of each class as a spreadsheet tab: System is one tab, Task another, CheckIn another. The relationships between them (a Task belongs to a System, a Subtask belongs to a Task) are also defined here.

● Functional Layer

TimestampMixin adds created_at/updated_at columns (auto-set) to every model. System is the top-level work domain with status enum (active/paused/archived), linked to Task, Priority, FocusBlock, AgentAssignment. Priority stores monthly score (1–10) per system with unique constraint on (system_id, year, month). Task has title, status (todo/in_progress/done/blocked), optional deadline, sort_order. Subtask belongs to Task and inherits priority from grandparent System. FocusBlock is a scheduled day for a System. AgentProgram is a named instruction set for an AI agent role. AgentAssignment links a program to a System. CheckIn is an end-of-day record. RebalanceProposal stores AI-generated plans as JSON with pending/approved/rejected status.

■ schemas.py

● Business Layer

While models.py defines what's stored in the database, schemas.py defines what travels over the network — the contract between frontend and backend. It defines what shape a request body must have, and what shape the API response will be.

● Functional Layer

Uses Pydantic v2. Every entity has Create (fields for creation), Update (all optional, for PATCH), and Read (full response including computed fields) variants. Computed fields: SystemRead.current_priority calls get_current_priority() at serialisation time. SubtaskRead.inherited_priority walks up Task→System. Intake schemas: IntakeStepRequest, IntakeStep, IntakeProposal, IntakeCommit. Report schemas: ReportSection (heading + items list), Report (title, summary, sections, generated_at). TodayView is the composite dashboard shape.

■ services.py

● Business Layer

The rule engine — decides which system to focus on today and which tasks are urgent or blocked. Fully deterministic (no AI), so the output is predictable and testable. Also contains the emit_event hook which the AI agents will listen to in future.

● Functional Layer

choose_focus_system(db) queries all active Systems with open tasks, ranks by priority score DESC then nearest deadline ASC, returns the top one. build_today(db) builds: focus_tasks (open tasks from focus system), upcoming (tasks with deadlines in next 7 days), flagged (overdue or blocked tasks). get_current_priority(db, system_id) fetches the Priority row for (system, current year+month). get_inherited_priority(db, subtask_id) follows Subtask→Task→System→Priority. emit_event() is a stub hook for future real-time agent triggers.

■ reports.py

● Business Layer

Generates all management reports — weekly, monthly, on-demand, and the morning briefing. Fully deterministic: queries the database and computes statistics with no AI involvement. The data is real; the narrative is generated from templates.

● Functional Layer

weekly_report(db) counts open/done tasks per system, finds overdue items, lists upcoming deadlines within 7 days. Sections: 'Behind' (overdue), 'Coming up' (imminent deadlines), 'Completion' (% done per system). monthly_report(db) looks at the whole month: completion %, missed deadlines, highest priority systems. on_demand_report(db) is a quick status snapshot: open task count per system, any overdue items. morning_briefing(db) calls choose_focus_system, lists flagged items, formats a short push-notification-ready summary. generated_at is an ISO timestamp in every report.

SECTION 2 — BACKEND: AI AGENTS

■ agents/llm.py

● Business Layer

The translator between MindAnchor and Claude (Anthropic's AI). Has two modes: a 'stub' mode that uses pure logic (no internet, no cost, always works), and a 'real' mode that calls the actual Claude API. The app automatically picks the right mode depending on whether an API key is present.

● Functional Layer

LLMClient protocol defines one method: propose(context: dict) -> dict. StubLLM is a deterministic planner: sorts open tasks by deadline, prepends a 'Prep:' subtask if any deadline is <=3 days away, returns JSON with summary and actions. Used in tests and when ANTHROPIC_API_KEY is empty. AnthropicLLM lazily imports anthropic, sends a strict-JSON prompt to claude-opus-4-8, parses and returns the JSON payload. get_llm() factory returns AnthropicLLM if API key is set, otherwise StubLLM.

■ agents/orchestrator.py

● Business Layer

Coordinates between the database, the AI, and the approval workflow. When you click 'Rebalance' on a system, it snapshots that system's state, asks the AI for a plan, validates the plan, and saves it as a 'pending' proposal for you to review. Nothing is applied until you approve.

● Functional Layer

build_context(db, system_id) assembles a dict snapshot: name, priority score, list of tasks with deadlines, statuses, subtask counts. propose_for_system(db, system_id) calls build_context, calls get_llm().propose(context), validates each action against an allow-list (reorder and add_pretask only — no delete, no cross-system actions), drops malformed actions silently, stores a RebalanceProposal with status=pending. apply_proposal executes approved actions: reorder updates sort_order, add_pretask creates a new Task at front. decide_proposal sets status to approved/rejected with decided_at timestamp and idempotency guard.

■ agents/intake.py

● Business Layer

When you want to add a new work System, this runs a conversational interview to understand your goals, constraints, and timeline. At the end it proposes an entire System with tasks and subtasks — which you review and approve before anything is saved.

● Functional Layer

StubIntake runs a fixed 6-question sequence offline (name, purpose, goals, constraints, dependencies, deadline) then generates a deterministic starter task tree. AnthropicIntake sends the full conversation history to Claude each call; Claude decides the next question or returns {done: true, proposal} when it has enough information. get_intake() factory selects by API key presence. Both share the same response shape: IntakeStep with {done: bool, question?: str, proposal?: IntakeProposal}.

SECTION 3 — BACKEND: API ROUTES

■ main.py

● **Business Layer**

The entry point of the server — the file that starts everything. Assembles all the individual routers into one running application and configures CORS and authentication.

● **Functional Layer**

Creates the FastAPI app instance. Adds CORSMiddleware using cors_origins from settings. Registers the auth router without the JWT dependency (it IS the login endpoint). Registers all other routers WITH dependencies=[Depends(get_current_user)] — this single line locks every route in that router. Two public endpoints: GET / (app info) and GET /health (liveness probe for Render health checks).

■ api/auth.py

● **Business Layer**

The login door. You send your password, you get back a token. That token is your key for every other request for the next 7 days.

● **Functional Layer**

POST /auth/login accepts {password: str}, calls verify_password() against settings.password_hash. On success, calls create_access_token() and returns {access_token, token_type: 'bearer'}. Returns HTTP 401 on wrong password with no detail about why — deliberate security practice.

■ api/systems.py

● **Business Layer**

Systems are the top-level containers for all work — departments or life areas (Health, Career, Side project). Handles creating, reading, updating, and archiving them, plus setting monthly priority scores.

● **Functional Layer**

GET /systems returns all systems with current priority computed. POST /systems creates from SystemCreate schema. GET/PATCH/DELETE /systems/{id} for single-system CRUD with cascade delete. GET/PUT /systems/{id}/priorities for upsert (create or update) the score for a given year+month.

■ api/tasks.py

● **Business Layer**

Tasks are the units of work within a System. Also handles Subtasks (smaller steps within a task). A subtask automatically inherits its grandparent system's priority.

● **Functional Layer**

GET /tasks?system_id= returns filtered task list with subtasks embedded. POST /tasks creates a task with auto-incremented sort_order. PATCH/DELETE /tasks/{id} for updates and cascade deletes. Full CRUD for /subtasks with inherited_priority in the Read response.

■ api/dashboard.py

● **Business Layer**

The 'Today' view — answers: What should I work on right now? What deadlines are coming? What is stuck or overdue?

Functional Layer

GET /dashboard/today calls build_today(db) from services.py and returns TodayView containing: focus_system (highest-priority active system), focus_tasks (open tasks in that system), upcoming (tasks with deadlines in next 7 days), flagged (overdue or blocked tasks across all systems).

■ api/calendar.py

Business Layer

Focus blocks are scheduled days where you commit to working on a specific system. This router manages your calendar of those commitments.

Functional Layer

GET /focus-blocks?start=&end;= for date-range filtered list. POST /focus-blocks creates a block for a given day, optionally linked to a system with a note. PATCH/DELETE /focus-blocks/{id} for update and removal.

■ api/checkins.py

Business Layer

At the end of each day you record which tasks you completed and any notes. This closes the daily loop and marks those tasks as done.

Functional Layer

GET /check-ins returns history most recent first. POST /check-ins creates a CheckIn record with {notes, completed_task_ids} and sets status=done on each task ID in the list.

■ api/rebalance.py

Business Layer

The human-in-the-loop gate for AI planning. You request a rebalance, the AI produces a proposal, you approve or reject it. The AI never acts unilaterally.

Functional Layer

POST /systems/{id}/rebalance calls propose_for_system() and returns the new pending proposal. GET /rebalance-proposals for filtered list. POST /rebalance-proposals/{id}/approve calls decide_proposal(approved) then apply_proposal() to execute changes. POST /rebalance-proposals/{id}/reject calls decide_proposal(rejected) with no changes applied.

■ api/intake.py

Business Layer

Powers the conversational new-system wizard. Each call advances the conversation one step until the AI proposes a full System with tasks.

Functional Layer

POST /intake/next accepts {history: [{question, answer}...]}, returns {done: false, question: str} or {done: true, proposal: IntakeProposal}. POST /intake/commit accepts the approved proposal, creates the System, all Tasks, and all Subtasks in a single database transaction.

■ api/reports.py

Business Layer

Provides the four pre-built reports. All are read-only — they query and compute, never modify data.

● **Functional Layer**

GET /reports/weekly, /reports/monthly, /reports/on-demand, /reports/morning-briefing — each calls the corresponding function from reports.py. All return Report schema: {title, summary, sections: [{heading, items}], generated_at}.

■ api/agents.py

● **Business Layer**

Agent Programs are reusable instruction sets defining how an AI agent should behave. Assignments link those programs to specific Systems.

● **Functional Layer**

GET/POST /agent-programs for list and create. PATCH/DELETE /agent-programs/{id} for update/delete. GET /agent-assignments lists all system–agent pairings. PUT /agent-assignments assigns a program to a system. PATCH/DELETE /agent-assignments/{id} for update/removal.

SECTION 4 — FRONTEND

■ types.ts

● Business Layer

The shared vocabulary of the app — defines in TypeScript all the data shapes the frontend knows about. If the backend and frontend shapes drift apart, TypeScript catches it at build time before it reaches users.

● Functional Layer

Enums: WorkStatus (todo/in_progress/done/blocked), SystemStatus (active/paused/archived), ProposalStatus (pending/approved/rejected). Interfaces mirror backend Read schemas 1-to-1: System, Task, Subtask, FocusBlock, CheckIn, TodayView, RebalanceProposal, ProposalAction, IntakeStep, IntakeProposal, Report, ReportSection.

■ api.ts

● Business Layer

The only place in the frontend that talks to the backend. Every network call goes through here. Handles attaching the login token to requests, and automatically redirects to the login screen on 401.

● Functional Layer

Token storage: getToken(), setToken(), clearToken() read/write/clear localStorage key 'ma_token'. login(password) posts to /auth/login, stores the returned JWT on success. request() core fetch wrapper adds Authorization: Bearer header, on 401 clears token and redirects to /login, on 204 returns undefined. The api object contains typed methods for every backend endpoint organised by domain.

■ main.tsx

● Business Layer

The entry point of the React application. Defines which URL path shows which page — the routing table of the app.

● Functional Layer

Uses React Router v6 createBrowserRouter + RouterProvider. App is the root layout. Routes: / → Dashboard, /systems → Systems, /calendar → Calendar, /proposals → Proposals, /intake → Intake, /reports → Reports. Wrapped in React.StrictMode.

■ App.tsx

● Business Layer

The permanent shell around every page — navigation sidebar (desktop), top nav bar (mobile), logo, Sign out button. Also acts as the route guard: if no token exists, the login screen is shown instead of any page content.

● Functional Layer

Route guard: useState(() => Boolean(getToken())). If false, renders Login component. Sidebar: 240px fixed left column with NavLink items, active link gets emerald gradient highlight. Top bar: horizontally scrollable nav strip on mobile with scrollbar-none. SignOutButton: calls clearToken(), navigates to /, reloads. Ambient background: fixed div with radial gradients giving the dark theme depth.

■ Login.tsx

● Business Layer

The login screen — the first thing a visitor sees. Styled with the Cosmos design system (dark glassmorphism aesthetic). One field: the owner password.

Functional Layer

Controlled form with password, loading, and error states. On submit: calls login(password), then calls onSuccess() prop which sets authed=true in App.tsx, re-rendering the full app. On failure: shows 'Incorrect password. Try again.' in red. Uses autoFocus on the password field and disabled button while loading.

■ Dashboard.tsx

Business Layer

The 'Today' page — shows the system to focus on, open tasks within it, upcoming deadlines across all systems, and blocked or overdue items. End-of-day check-in form to record completions.

Functional Layer

useEffect on mount calls api.today() to load TodayView. Renders three Card sections: Focus (system + task checkboxes), Upcoming (deadline list), Flagged (overdue/blocked). Check-in form tracks which task IDs are checked, collects notes, calls api.createCheckIn() on submit. Shows StatusBadge per task with colour-coded work status.

■ Systems.tsx

Business Layer

Management page for all work systems. Create systems, set monthly priority, manage tasks and subtasks, request AI rebalance proposals.

Functional Layer

Loads all systems on mount, each is collapsible. Priority selector: month/year dropdowns + numeric score input → api.setPriority(). Task list per system with inline add form, status toggle, deadline edit, delete. Subtask expand loads subtasks on demand via api.listSubtasks(taskId). Rebalance button calls api.requestRebalance(systemId) and shows a success notice linking to Proposals page.

■ Calendar.tsx

Business Layer

Scheduling page — block out days for specific systems so your calendar reflects your actual priorities. Simple agenda view (chronological list, no grid).

Functional Layer

Loads focus blocks and systems on mount. Add form: date picker, system dropdown (optional), note field → api.createFocusBlock(). Agenda grouped by day using reduce over sorted blocks. Delete button per block → api.deleteFocusBlock(id).

■ Proposals.tsx

Business Layer

The approval queue for AI suggestions. Review proposals and decide whether to apply or discard them. Embodies the core design principle: AI proposes, human decides.

Functional Layer

Loads pending proposals on mount via api.listProposals('pending'). Resolves task titles by loading all tasks and building an id→title lookup map. Each proposal shows: system name, AI summary, actions in plain English. Approve → api.approveProposal(id); Reject → api.rejectProposal(id). Both remove the proposal from the list.

■ Intake.tsx

● Business Layer

Conversational wizard for creating a new System. Back-and-forth conversation with the AI, ending with a structured proposal (system + tasks + subtasks) to review before committing.

● Functional Layer

history state is array of {question, answer} pairs. Each 'Send' calls api.intakeNext(history), returns either next question or {done: true, proposal}. If not done: appends answered pair to history, shows next question. If done: renders proposal review with task tree. 'Create system' calls api.intakeCommit(proposal), navigates to /systems.

■ Reports.tsx

● Business Layer

Analytics and reporting page — shows how each system is performing: what's behind, what's on track, completion percentages. Also provides a morning briefing preview with browser notifications.

● Functional Layer

SlidingTabBar: animated tab selector (weekly/monthly/on-demand) with spring-transition gradient pill driven by measuring offsetLeft with useRef. StatStrip: three KPI cards from real data (Behind count, Coming up count, avg completion %). ReportSection: coloured left border and dots by semantic meaning (behind=red, coming up=amber, completion=green). MetricText: splits % strings and colours by threshold (>=80 green, >=40 amber, <40 red). FreshnessBadge: monospace timestamp, green if <1hr old. SkeletonReport: shimmer placeholder while data loads. Morning briefing section fires notification via Service Worker.

SECTION 5 — INFRASTRUCTURE & CONFIG

■ Dockerfile

● Business Layer

Blueprint for packaging the backend into a portable container. Ensures that no matter which server runs MindAnchor, it always runs with the exact same Python version, dependencies, and startup sequence.

● Functional Layer

Multi-stage build. Builder stage: installs gcc and libpq-dev to compile psycopg2, installs all Python packages into /install. Runtime stage: starts from clean python:3.11-slim, copies only installed packages (not build tools), copies application code. Runs as non-root user (appuser) — security best practice. ENV PORT=8080. CMD runs: alembic upgrade head (applies pending migrations automatically on every container start), then uvicorn app.main:app (starts the API server).

■ requirements.txt

● Business Layer

The exact list of Python packages with pinned versions. Pinning ensures the production deploy uses the same library versions as development — preventing 'works on my machine' surprises.

● Functional Layer

Web framework: fastapi 0.115.6, uvicorn[standard]. Database: sqlalchemy 2.0.36, alembic 1.14.0, psycopg2-binary. Validation: pydantic 2.10.4, pydantic-settings 2.7.0. Auth: python-jose[cryptography] 3.3.0, passlib[bcrypt] 1.7.4, bcrypt 4.0.1 (pinned because passlib 1.7.4 breaks with bcrypt >=4.1). AI: anthropic 0.42.0. HTTP client: httpx 0.28.1. Dev/test: pytest, ruff.

■ vite.config.ts

● Business Layer

Configures how the frontend is built and how it behaves in development. Enables PWA capabilities — offline support, installability on mobile, and caching.

● Functional Layer

React plugin enables JSX transform and fast refresh. Dev server proxy: /api/* forwarded to http://localhost:8000 — frontend never hardcodes the backend URL or deals with CORS in dev. VitePWA: registerType autoUpdate, Workbox precaches all built assets, runtime StaleWhileRevalidate cache for /api/* (5-min max age, 50 entry limit). Manifest: name 'MindAnchor', standalone display, full icon set (192px, 512px, maskable), orientation, category 'productivity'.

■ vercel.json

● Business Layer

Tells Vercel how to handle incoming requests. The critical job is proxying API calls from the browser to the Render backend, since they live on different domains.

● Functional Layer

Rewrite rule: /api/:path* → https://mindanchor-api.onrender.com/:path* — rewrites happen at the Vercel edge server-side, bypassing browser CORS entirely. Headers: service worker served with no-cache so the browser always checks for updates. Built assets served with immutable cache headers (1-year, content-hashed filenames).

■ index.html

Business Layer

The single HTML page the browser loads. In a React SPA there is only ever one HTML file — React takes over from here and renders everything else in JavaScript.

Functional Layer

Sets lang, charset, viewport with viewport-fit=cover for iPhone notch. PWA meta tags: theme-color, apple-mobile-web-app-capable, apple-mobile-web-app-status-bar-style. Google Fonts: preconnects and loads Inter (400–800 weights) and JetBrains Mono (600 weight). Icon links: favicon.svg, favicon.png (32px), apple-touch-icon.png (180px). div#root is the React mount point.

■ index.css

Business Layer

The global stylesheet — the visual design language. Defines the colour tokens, spacing, and component styles that give MindAnchor its 'Cosmos' dark sci-fi aesthetic. The design system written in CSS.

Functional Layer

:root CSS variables: Background scale --color-void (#03040a) through --color-hull (#0f1630). Signal colours: --color-signal-ok (emerald), --color-signal-warn (amber), --color-signal-crit (red), --color-signal-idle (slate). --glass-bg and --glass-border for glassmorphism panels. --glow-ok/warn/crit/ui box-shadow glow presets. --ease-spring cubic-bezier(0.34,1.56,0.64,1) for bouncy transitions. Component classes: .glass-panel (backdrop-blur frosted glass), .metric (JetBrains Mono tabular numerals), .cosmos-btn-primary (emerald gradient with glow), .cosmos-btn-secondary (violet ghost button), .skeleton-bone (shimmer loader). Keyframes: fade-up, shimmer. prefers-reduced-motion block disables all animations for accessibility.

■ .github/workflows/ci.yml

Business Layer

Automates quality checks and deployment. Every code push automatically checks style, runs tests, builds the frontend, and deploys to Render and Vercel. Humans don't need to remember any of these steps.

Functional Layer

Five jobs in dependency order: (1) Backend lint+test: ruff check then pytest. (2) Frontend build: npm ci + tsc -b + vite build. (3) Docker build+push to Google Artifact Registry using Workload Identity Federation. (4) Cloud Run deploy + alembic upgrade head as a Cloud Run Job. (5) Vercel deploy of pre-built frontend dist.

■ tests/

Business Layer

The automated test suite. Every code change triggers these tests in CI to catch regressions before they reach production. All tests use in-memory SQLite — no live PostgreSQL or internet needed.

Functional Layer

conftest.py has two autouse fixtures: _force_offline_agents (sets ANTHROPIC_API_KEY='' so tests never call real Claude) and _bypass_auth (overrides get_current_user to return 'owner' without a token). Tests: test_health.py (smoke tests), test_crud.py (data model + relationships), test_dashboard.py (focus selection logic), test_rebalance.py (propose→approve→apply flow), test_intake.py (conversational interview + commit), test_reports.py (report structure + content), test_auth.py (login success/failure, 401 without token).